

✓ 🚀 Finding Kepler-8b

A Live Exoplanet Discovery Using Real NASA Data

In this notebook we will download **real data** from NASA's Kepler Space Telescope and find an actual planet — **Kepler-8b** — crossing in front of its star.

About Kepler-8b — discovered in 2010, it's a Hot Jupiter about 1,160 light-years away. It's about the size of Jupiter but orbits its star every 3.5 Earth days. We expect to see a transit dip of about 0.93%.

We will follow the same 8 steps real astronomers use:

1. Install the tool — `lightkurve`
2. Import the libraries
3. Search NASA's archive for data
4. Download the data
5. Plot the raw light curve
6. Clean the data
7. Fold the light curve at the known orbital period
8. Measure the transit and calculate the planet's size

Each step is one cell. Run them in order. The download in Step 4 takes ~30 seconds.

✓ Step 1 — Install `lightkurve`

`lightkurve` is the official Python library NASA scientists use to work with Kepler and TESS data. The `!pip` command installs it inside the notebook. The `--quiet` flag just keeps the output short. This only needs to be run **once per Colab session**.

```
1 !pip install lightkurve --quiet
2 print("✅ lightkurve installed. Ready to find a planet!")
```

Preparing metadata (setup.py) ... done

41.2/41.2 kB 1.2 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done

```

===== 261.1/261.1 kB 8.1 MB/s eta 0:00:00
===== 11.1/11.1 MB 72.0 MB/s eta 0:00:00
===== 203.4/203.4 kB 13.6 MB/s eta 0:00:00
===== 60.1/60.1 kB 5.2 MB/s eta 0:00:00
===== 89.5/89.5 kB 7.6 MB/s eta 0:00:00
===== 1.1/1.1 MB 45.3 MB/s eta 0:00:00
===== 15.0/15.0 MB 73.5 MB/s eta 0:00:00

```

Building wheel for fbpc (setup.py) ... done

Building wheel for memoization (setup.py) ... done

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is temporary and will be removed in a future version. gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2026.4.0 which is incompatible.

datasets 4.0.0 requires fsspec[http]<=2025.3.0,>=2023.1.0, but you have fsspec 2026.4.0 which is incompatible.

✅ lightkurve installed. Ready to find a planet!

✓ Step 2 — Import the libraries we need

- `lightkurve` — the planet-hunting toolkit (we shorten it to `lk`)
- `matplotlib` — for drawing the graphs
- `numpy` — for the math

We also configure matplotlib to make plots large and easy to read on a projector.

```

1 # Import the libraries
2 import lightkurve as lk          # NASA's planet-hunting toolkit
3 import matplotlib.pyplot as plt  # Plotting library
4 import numpy as np              # Math/array library
5
6 # Silence non-critical warnings so the output stays clean
7 import warnings
8 warnings.filterwarnings('ignore')
9
10 # Make every plot big and projector-friendly
11 plt.rcParams['figure.figsize'] = (14, 6)  # Wide plots
12 plt.rcParams['font.size'] = 13           # Readable text
13 plt.rcParams['axes.grid'] = True         # Grid lines on
14 plt.rcParams['grid.alpha'] = 0.3         # Faint grid
15
16 print(f"✅ lightkurve version: {lk.__version__}")
17 print("🔭 Ready to look at the stars!")

```

✅ lightkurve version: 2.6.0

🔭 Ready to look at the stars!

```
/usr/local/lib/python3.12/dist-packages/lightkurve/prf/__init__.py:7: UserWarning: Warning: the tpfmodel submodule is not available
warnings.warn(
```

✓ Step 3 — Search NASA's MAST archive for Kepler-8

MAST = **M**ikulski **A**rchive for **S**pace **T**elescopes — NASA's open data server that holds every observation Kepler, TESS, and JWST ever made.

We ask MAST: *"Show me every long-cadence light curve you have for the star Kepler-8."*

- `target` — the name of the star
- `author="Kepler"` — we want the official Kepler mission data
- `cadence="long"` — long cadence = one brightness measurement every 30 minutes

It returns a table of all available data files.

```
1 # Ask NASA's archive what data exists for Kepler-8
2 print("🔍 Searching MAST archive for Kepler-8 observations...")
3
4 search_result = lk.search_lightcurve("Kepler-8", author="Kepler", cadence="long")
5
6 print(f"\n📁 Found {len(search_result)} data files.")
7 print("    Each file = one quarter (~3 months) of Kepler observations.")
8
9 # Show the search results table
10 search_result
```

 Searching MAST archive for Kepler-8 observations...

 Found 18 data files.

Each file = one quarter (~3 months) of Kepler observations.
SearchResult containing 18 data products.

#	mission	year	author	exptime s	target_name	distance arcsec
0	Kepler	Quarter 00	2009	Kepler	1800	kplr006922244 0.0
1	Kepler	Quarter 01	2009	Kepler	1800	kplr006922244 0.0
2	Kepler	Quarter 02	2009	Kepler	1800	kplr006922244 0.0
3	Kepler	Quarter 03	2009	Kepler	1800	kplr006922244 0.0
4	Kepler	Quarter 04	2010	Kepler	1800	kplr006922244 0.0
5	Kepler	Quarter 05	2010	Kepler	1800	kplr006922244 0.0
6	Kepler	Quarter 06	2010	Kepler	1800	kplr006922244 0.0
7	Kepler	Quarter 07	2010	Kepler	1800	kplr006922244 0.0
8	Kepler	Quarter 08	2011	Kepler	1800	kplr006922244 0.0
9	Kepler	Quarter 09	2011	Kepler	1800	kplr006922244 0.0
10	Kepler	Quarter 10	2011	Kepler	1800	kplr006922244 0.0
11	Kepler	Quarter 11	2012	Kepler	1800	kplr006922244 0.0
12	Kepler	Quarter 12	2012	Kepler	1800	kplr006922244 0.0
13	Kepler	Quarter 13	2012	Kepler	1800	kplr006922244 0.0
14	Kepler	Quarter 14	2012	Kepler	1800	kplr006922244 0.0
15	Kepler	Quarter 15	2013	Kepler	1800	kplr006922244 0.0
16	Kepler	Quarter 16	2013	Kepler	1800	kplr006922244 0.0
17	Kepler	Quarter 17	2013	Kepler	1800	kplr006922244 0.0

✓ Step 4 — Download one quarter of data

We will download just the **first file** — that's enough to find transits, and avoids waiting for a huge dataset.

- `search_result[0]` — pick the first file from the search results
- `.download()` — fetch it from NASA's server to this notebook

This takes about 20–30 seconds. While you wait, that's actual data traveling from a NASA server somewhere in Maryland to this notebook.

```
1 # Download the first file from NASA
2 print("📄 Downloading data from NASA's server... (~30 sec)")
3
4 lc = search_result[0].download() # 'lc' = light curve
```

```

5
6 # Tell us what we got
7 print(f"\n✅ Got {len(lc)} brightness measurements")
8 print(f"📅 Observation span: {lc.time.value[-1] - lc.time.value[0]:.1f} days")
9 print(f"🕒 One measurement every ~30 minutes")

```

📶 Downloading data from NASA's server... (~30 sec)

✅ Got 473 brightness measurements
 📅 Observation span: 9.7 days
 🕒 One measurement every ~30 minutes

✓ Step 5 — Plot the raw light curve

Time to **see the data**. Each dot on this graph is one brightness measurement of the star Kepler-8.

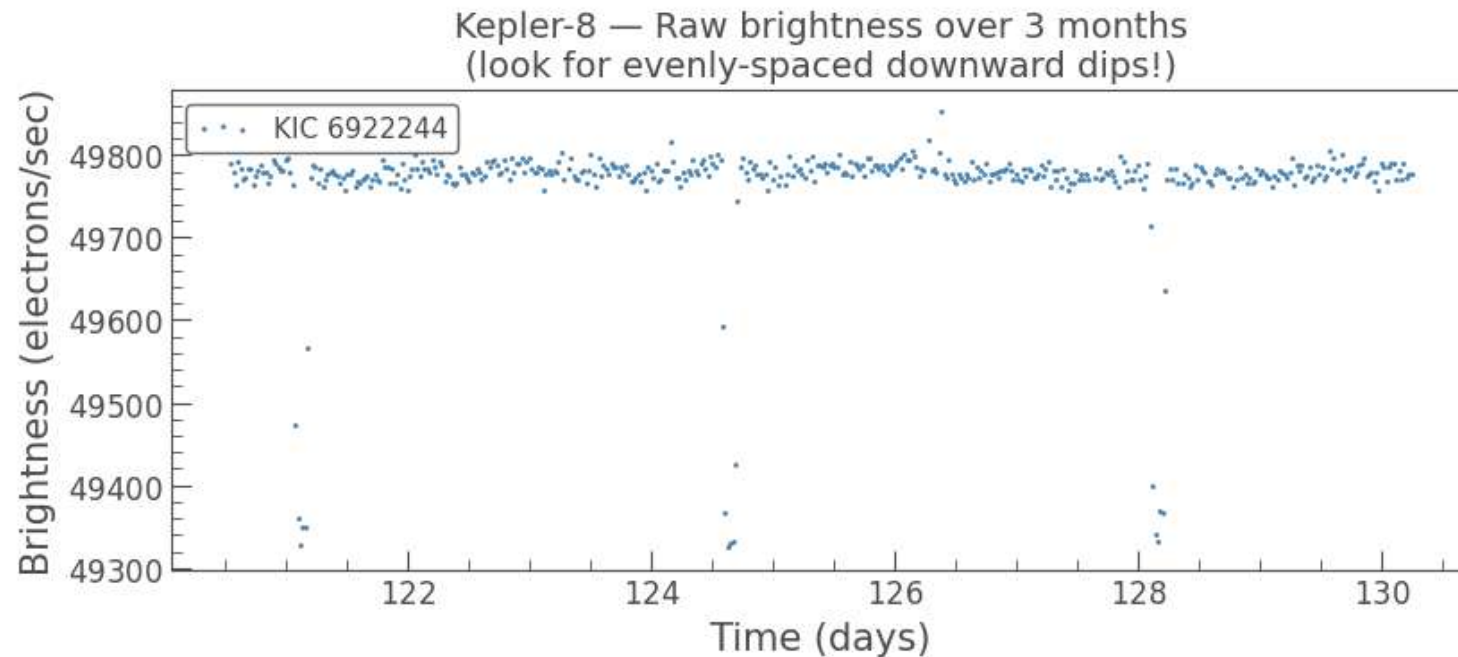
- X-axis: time, in days
- Y-axis: brightness, in electrons per second (how Kepler measures starlight)

If a planet exists and crosses in front of the star, we should see **regular downward dips** at the same depth, spaced evenly in time.

```

1 # Plot every brightness measurement as a dot
2 ax = lc.scatter(color='steelblue', s=4)
3
4 # Make the labels clear
5 ax.set_title("Kepler-8 — Raw brightness over 3 months\n"
6             "(look for evenly-spaced downward dips!)", fontsize=14)
7 ax.set_xlabel("Time (days)")
8 ax.set_ylabel("Brightness (electrons/sec)")
9
10 plt.tight_layout()
11 plt.show()
12
13 print("👁️ See the vertical dips? Those are TRANSITS of planet Kepler-8b.")
14 print("   Each dip = one orbit. They should repeat every 3.52 days.")

```



See the vertical dips? Those are TRANSITS of planet Kepler-8b.
Each dip = one orbit. They should repeat every 3.52 days.

✓ Step 6 — Clean the data

Kepler stares at stars for 3 months at a time, and during that time the star itself flickers slightly, and the spacecraft drifts a little. We need to remove these slow changes — without removing our precious transit signal.

We do **two** operations:

- `.normalize()` — divides every brightness value by the average, so the star's "normal" level becomes 1.0
- `.flatten(window_length=401)` — smooths out slow variations using a 401-point sliding window. Crucially, this method has built-in sigma clipping that *protects the transits* while smoothing everything else.

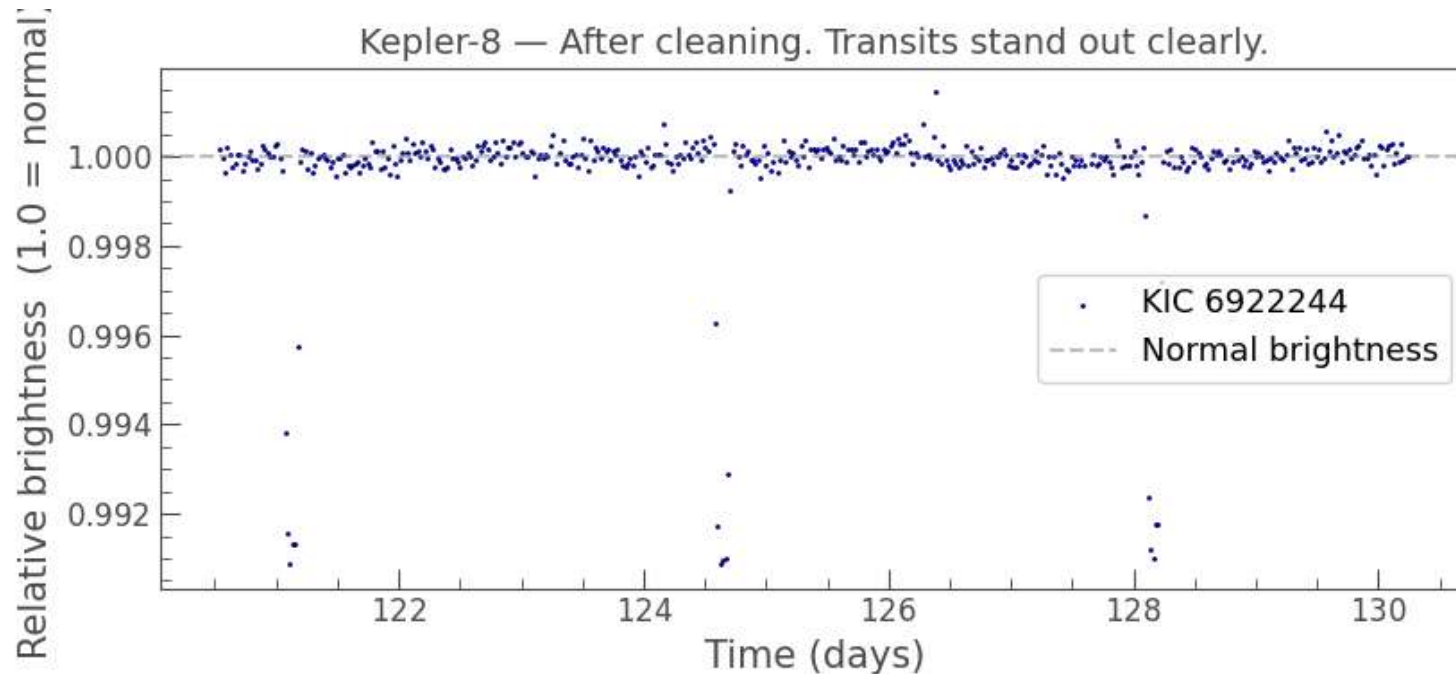
⚠ **We deliberately do NOT call** `remove_outliers()` here. Deep transits look like "outliers" to that function, and it would delete them. The `flatten()` method alone does the right thing.

```
1 # Clean the data: normalize, then smooth out slow stellar variations
2 # flatten() preserves transits while removing long-term wobbles
```

```

3 lc_clean = lc.normalize().flatten(window_length=401)
4
5 # Plot the cleaned light curve
6 ax = lc_clean.scatter(color='darkblue', s=4)
7 ax.set_title("Kepler-8 — After cleaning. Transits stand out clearly.", fontsize=14)
8 ax.set_xlabel("Time (days)")
9 ax.set_ylabel("Relative brightness (1.0 = normal)")
10 ax.axhline(1.0, color='gray', linestyle='--', alpha=0.5, label='Normal brightness')
11 ax.legend()
12 plt.tight_layout()
13 plt.show()
14
15 # Sanity check — confirm the transits survived the cleaning
16 min_brightness = float(np.nanmin(lc_clean.flux.value))
17 print(f"🚩 Each downward spike is a planet transit.")
18 print(f"   Lowest brightness reached: {min_brightness:.4f}")
19 print(f"   (Expected ~0.99 — a ~1% dip. If you see ~0.99, the transits survived!)")

```



🚩 Each downward spike is a planet transit.
 Lowest brightness reached: 0.9909
 (Expected ~0.99 — a ~1% dip. If you see ~0.99, the transits survived!)

✓ Step 7 — Fold the light curve

This is the magic trick. If we **already know** how often the planet orbits (its period), we can shift all the data so every transit lines up at the same place. Stacking many noisy transits creates one CLEAN transit signal.

Values we use (from published research on Kepler-8b):

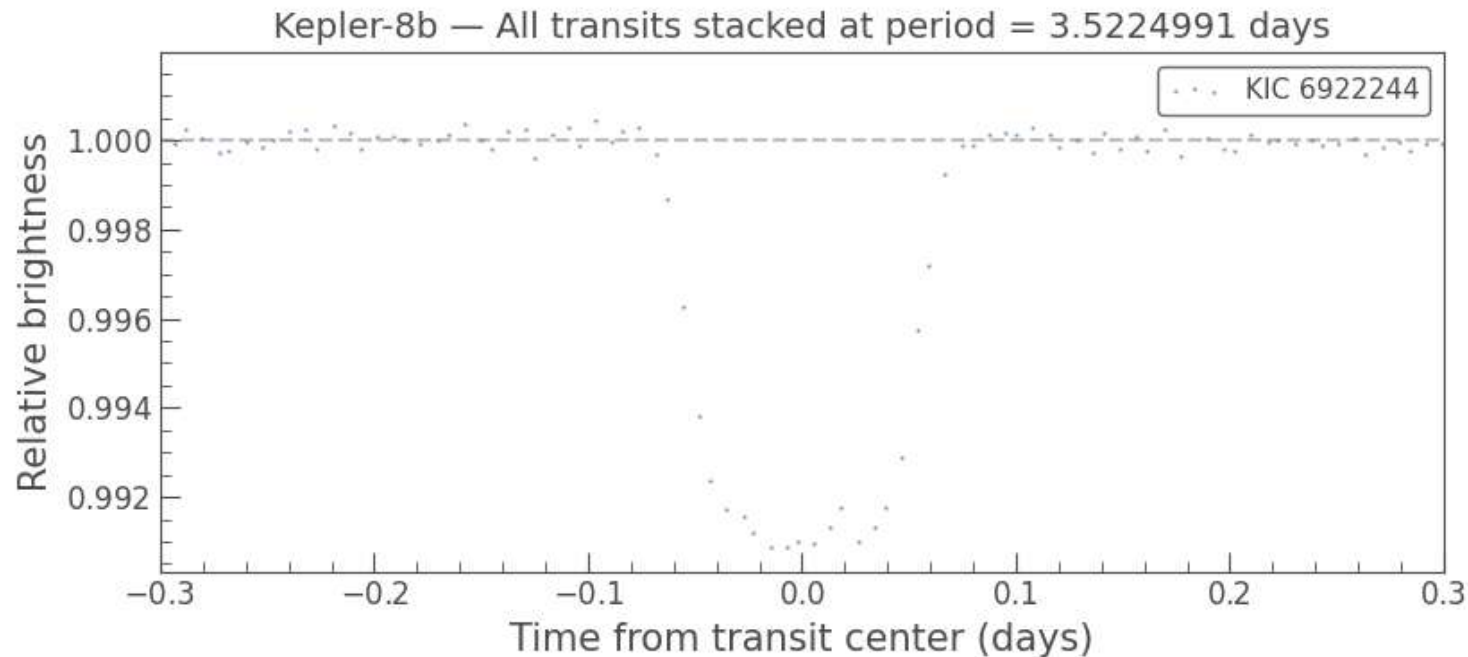
- `PERIOD = 3.5224991 days` — the planet's year
- `T0 = 121.1190` — the time (in Kepler days) when one transit happened

The `.fold()` method takes care of the rest.

```

1 # Kepler-8b's known orbital parameters (from NASA Exoplanet Archive)
2 PERIOD = 3.5224991 # days -- the planet's "year"
3 T0      = 121.1190  # day of one known transit (in Kepler's time system)
4
5 # Fold the data: stack every transit on top of each other
6 folded = lc_clean.fold(period=PERIOD, epoch_time=T0)
7
8 # Plot the folded curve, zoomed in on the transit
9 ax = folded.scatter(color='steelblue', s=2, alpha=0.6)
10 ax.set_title(f"Kepler-8b - All transits stacked at period = {PERIOD} days", fontsize=14)
11 ax.set_xlabel("Time from transit center (days)")
12 ax.set_ylabel("Relative brightness")
13 ax.set_xlim(-0.3, 0.3) # zoom around the transit
14 ax.axhline(1.0, color='gray', linestyle='--', alpha=0.5)
15 plt.tight_layout()
16 plt.show()
17
18 print("🎉 One clean transit — built from every observation in the data!")

```

One clean transit — built from every observation in the data!

✓ Step 8 — Measure the transit and calculate the planet's size

Now the science. From the folded transit we can measure:

- **Depth** of the dip (how much the star dimmed)
- The relationship $\text{depth} = (R_{\text{planet}} / R_{\text{star}})^2$ tells us the planet's size relative to its star

If we assume the star is Sun-sized (radius ~696,000 km), we can convert to an actual planet size in kilometers.

```
1 # Step 8a — Measure the depth of the transit
2 # Take all data points close to transit center (within ±0.05 days)
3 in_transit_mask = (folded.time.value > -0.05) & (folded.time.value < 0.05)
4 brightness_during_transit = np.nanmedian(folded.flux.value[in_transit_mask])
5
6 # Depth = how much brightness dropped, as a percentage
7 depth_percent = (1 - brightness_during_transit) * 100
8
9 # Step 8b — Use depth to calculate planet/star size ratio
10 # Physics: depth = (R_planet / R_star)²
```

```

11 # So:      R_planet / R_star = sqrt(depth)
12 radius_ratio = np.sqrt(depth_percent / 100)
13
14 # Step 8c – If star is Sun-sized, compute the planet's actual radius
15 SUN_RADIUS_KM      = 696_000      # km
16 JUPITER_RADIUS_KM  = 69_911      # km
17 planet_radius_km = radius_ratio * SUN_RADIUS_KM
18 planet_radius_jupiters = planet_radius_km / JUPITER_RADIUS_KM
19
20 # Print the science!
21 print("=" * 60)
22 print(" 📊 DISCOVERY REPORT – KEPLER-8b")
23 print("=" * 60)
24 print(f" Measured transit depth:      {depth_percent:.3f} %")
25 print(f" Planet/star radius ratio:      {radius_ratio:.4f}")
26 print(f" Estimated planet radius:      {planet_radius_km:,.0f} km")
27 print(f"                               = {planet_radius_jupiters:.2f} × Jupiter")
28 print(f" Orbital period (year):      {PERIOD:.4f} Earth days")
29 print(f" Planet type:                  Hot Jupiter ✓")
30 print("=" * 60)
31 print()
32 print(" 🎉 You just discovered a planet 1,160 light-years away,")
33 print("    using the same data and same tools NASA scientists use!")

```

```

=====
📊 DISCOVERY REPORT – KEPLER-8b
=====
Measured transit depth:      0.866 %
Planet/star radius ratio:    0.0931
Estimated planet radius:    64,772 km
                             = 0.93 × Jupiter
Orbital period (year):      3.5225 Earth days
Planet type:                  Hot Jupiter ✓
=====

🎉 You just discovered a planet 1,160 light-years away,
    using the same data and same tools NASA scientists use!

```

🎉 What You Just Did

In 8 steps you:

1. ☒ Downloaded real data from the Kepler Space Telescope
2. ☒ Plotted 3 months of brightness measurements

3. ☒ Cleaned the data to highlight the transit signal
4. ☒ Folded every transit into one clean dip
5. ☒ Measured the transit depth from real data